

INC-001 - Apache Service Failure / Self-Healing

Terraform AWS High Availability Platform | Incident Testing and Recovery

Incident: Apache was deliberately stopped on one EC2 instance to simulate an application service failure and validate automated recovery.

1. Environment

Component	Configuration
Cloud	AWS - ap-southeast-2 (Sydney)
Compute	Amazon EC2 - Amazon Linux 2023
Web Service	Apache HTTP Server (httpd)
Load Balancing	Application Load Balancer + Target Group
Auto Scaling	project2-web-asg
ASG Capacity	Minimum 2 Desired 2 Maximum 3
Health Check	ELB / Target Group health check on /
Access Method	AWS Systems Manager Session Manager

2. What Happened

The platform was running two healthy EC2 instances behind an Application Load Balancer. To test resilience, the Apache service on one instance was deliberately stopped through AWS Systems Manager.

Once Apache stopped responding on HTTP port 80, the Target Group health check failed for that instance. The Application Load Balancer stopped sending normal application traffic to the unhealthy target, while the Auto Scaling Group detected the unhealthy instance and launched a replacement to restore the desired capacity.

3. Failure Simulation

The Apache service was stopped on one EC2 instance using:

```
sudo systemctl stop httpd
```

This simulated an application-level failure: the EC2 instance could still exist, but the web service was no longer responding.

4. Detection and Troubleshooting

Baseline check: Confirmed two EC2 instances were InService and Healthy before the test.

Service failure: Stopped Apache on one selected instance using Session Manager.

Target health: Observed the affected target transition to Unhealthy after failed health checks.

Load balancer behavior: Verified the ALB continued serving traffic through the remaining healthy target.

Auto Scaling response: Observed the ASG replace the unhealthy instance to maintain desired capacity.

Recovery validation: Confirmed the replacement instance became InService and Healthy and the Target Group returned to two healthy targets.

5. Key Diagnostic Evidence

Check	Observed Result
ASG before failure	2 instances - InService / Healthy
Apache on test instance	Stopped
Target Group	Affected target became Unhealthy
ALB routing	Traffic continued through healthy target
ASG recovery	Replacement EC2 instance launched
Final Target Group state	2 healthy targets
Final ASG state	Desired capacity restored

6. Resolution

No manual EC2 replacement was required. The high-availability design recovered automatically. The Target Group identified the failed web service, the ALB avoided the unhealthy target, and the Auto Scaling Group replaced the unhealthy instance to restore the configured capacity.

7. Recovery Flow

```
Apache stopped on EC2 instance
|
Target Group health check fails
|
Target marked Unhealthy
|
ALB routes traffic to healthy target
|
Auto Scaling Group replaces unhealthy instance
|
Replacement passes health checks
|
Target Group returns to 2 healthy targets
```

8. Incident Outcome

Validation	Result
Apache failure successfully simulated	PASS
Target Group detected unhealthy service	PASS
ALB maintained service through healthy target	PASS
ASG initiated replacement	PASS
Desired capacity restored	PASS
Replacement instance became healthy	PASS
Target Group returned to 2 healthy targets	PASS

9. Lessons Learned

This test demonstrated the difference between instance availability and application availability. An EC2 instance can still exist while its web service has failed. Target Group health checks allow the platform to detect that application failure, while the ALB and Auto Scaling Group work together to maintain service availability and restore healthy capacity.

Suggested GitHub evidence: include screenshots showing the healthy baseline, the unhealthy target after Apache was stopped, the ASG replacement activity, and the final Target Group state with two healthy targets.